Ingeniería y Calidad de Software — Segundo parcial

Resumen unificado de los temas 1 al 7

Índice: 1. Testing de software · 2. Testing ágil y TDD · 3. Lean · 4. Kanban y métricas · 5. Calidad de proceso y producto · 6. Aseguramiento y mejora de procesos · 7. Revisiones técnicas · 8. Cuadros integradores.

---

1. Testing de software

1.1. Qué es y cuál es su propósito

El **testing** o prueba de software es una actividad sistemática que intenta **descubrir defectos** y aportar información sobre el comportamiento y los riesgos de un producto. El enfoque que utiliza la cátedra lo presenta como un **proceso destructivo**: no se prueba para demostrar que el software carece de defectos, sino con la actitud crítica de intentar encontrarlos. Por eso, desde esta perspectiva, una prueba que revela un defecto cumple especialmente bien su objetivo.

El testing forma parte de las actividades relacionadas con la calidad, pero **no equivale a todo el aseguramiento de calidad**, ni reemplaza las revisiones, la gestión de configuración o la mejora de procesos. Las pruebas permiten obtener evidencia; por sí solas no garantizan la ausencia de defectos. El material remarca que el testing de un sistema confiable puede consumir una porción significativa del costo total del desarrollo.

Error, defecto y falla

En la explicación de clase, el **error** es una equivocación que puede descubrirse en la misma etapa en que se produce mediante actividades de revisión; el **defecto** es un problema que escapó de esa etapa y se trasladó a una posterior. Las lecturas técnicas también distinguen la **falla**, manifestación observable de un comportamiento incorrecto durante la ejecución. Estas nociones están relacionadas, pero no son intercambiables en todos los contextos bibliográficos.

Ejemplo de aplicación: una regla de descuento mal interpretada en el diseño es un error que puede descubrirse al revisar el diseño. Si pasa al código, queda incorporada como defecto. Si una compra aplica un descuento equivocado al ejecutarse, se observa una falla.

Severidad y prioridad

Concepto	Pregunta que responde	Criterio
Severidad	¿Qué tan grave es el defecto?	Impacto técnico o funcional: bloqueante, crítico, mayor, menor, cosmético, según la clasificación usada.
Prioridad	¿Con qué urgencia debe corregirse?	Contexto del negocio, fecha de entrega, usuarios afectados, riesgos y compromisos.

No siempre coinciden: una falta ortográfica visible en una pantalla que se mostrará en una presentación inminente puede tener baja severidad y alta prioridad. Un defecto grave en una funcionalidad todavía no habilitada puede recibir una prioridad distinta según el contexto.

1.2. Principios y límites de las pruebas

- **No es posible probar exhaustivamente** todas las combinaciones de datos, estados y recorridos de un sistema real.
- Las pruebas **muestran presencia de defectos**, no demuestran de manera absoluta su ausencia.
- Es necesario definir **resultados esperados** antes de ejecutar un caso de prueba: sin un oráculo de comparación no se puede evaluar correctamente el resultado.
- Las pruebas deben considerar tanto **entradas válidas como inválidas**, situaciones normales y excepcionales.
- Encontrar defectos temprano reduce el costo de propagarlos a etapas posteriores.
- Los casos de prueba deben seleccionarse con criterio: se busca **máxima cobertura relevante con esfuerzo limitado**.
- Una concentración de defectos en ciertas áreas orienta dónde profundizar las pruebas.

La pregunta **«¿cuánto testing es suficiente?»** no se resuelve contando pruebas. Se consideran riesgo, criticidad, cobertura, tiempo, costo y evidencia de calidad. El material insiste en evitar el falso objetivo de probarlo todo.

1.3. Niveles de prueba

Los niveles amplían progresivamente el alcance del objeto que se prueba:

Nivel	Objeto y propósito	Responsable o participación habitual
Unitario / de componente	Una función, clase o componente aislado; comprueba su comportamiento puntual.	Principalmente desarrollo.
Integración	La interacción entre componentes, módulos, interfaces y servicios; busca errores de comunicación y contratos.	Desarrollo y/o testing.
Sistema	El sistema completo y sus requisitos funcionales y no funcionales.	Equipo de testing o QA, según organización.
Aceptación	Determinar si el sistema satisface las necesidades y criterios del cliente/usuario y genera confianza para su uso.	Cliente, usuario o representantes del negocio.

La **aceptación** no tiene como objetivo principal descubrir defectos, aunque puede encontrarlos; apunta a validar la adecuación del producto a lo requerido. Los niveles no deben confundirse con **tipos de pruebas** ni con **ambientes**.

1.4. Ambientes

Ambiente	Finalidad
Desarrollo	Construir, depurar y ejecutar pruebas locales, especialmente unitarias.
Pruebas / testing	Ejecutar casos planificados en un entorno controlado, registrar resultados y defectos.
Preproducción	Reproducir, en la medida de lo posible, condiciones semejantes a producción; verificar integraciones y cambios críticos antes de desplegar.
Producción	Entorno real utilizado por los usuarios; exige control de cambios, monitoreo y precaución.

El ambiente adecuado depende de qué se necesita verificar. Las pruebas críticas de integración o de despliegue pueden requerir **preproducción** para disminuir riesgos antes de liberar el sistema.

1.5. Casos de prueba y proceso de testing

Un **caso de prueba** describe qué se verifica, bajo qué condiciones, con qué datos, qué pasos se ejecutan y cuál es el **resultado esperado**. El resultado obtenido debe compararse con ese esperado; además se documenta el estado de ejecución y, si corresponde, el defecto observado.

Elementos útiles: identificador; requisito o característica cubierta; objetivo; precondiciones; datos de entrada; pasos; resultado esperado; resultado real; estado; evidencias. Una **suite** agrupa casos relacionados y un **ciclo de prueba** organiza su ejecución en un momento o versión del producto.

Proceso general: **planificar** alcance y criterios → **diseñar** casos y datos → **preparar** ambiente → **ejecutar** → **comparar y registrar** resultados → **informar** defectos → **reprobar** correcciones y realizar regresión cuando corresponda → **evaluar** criterios de cierre.

Ejemplo de aplicación: para un formulario que acepta edades enteras entre 18 y 65, documentar casos con 17, 18, 65 y 66, además de entradas inválidas. No alcanza con decir «probar edades»: deben constar los resultados esperados.

1.6. Estrategias y métodos de prueba

El material propone estrategias **complementarias** para dirigir el esfuerzo de testing:

Pruebas basadas en especificación / caja negra

Se derivan de requisitos, entradas, salidas y reglas del negocio, **sin depender de conocer la estructura interna** del programa. Permiten evaluar si el sistema hace lo que debe hacer.

- **Partición de equivalencia:** dividir el dominio de entrada en clases que deberían comportarse de forma equivalente y seleccionar representantes de cada clase.
- **Análisis de valores límite:** probar extremos y valores cercanos a los bordes, porque suelen concentrar errores.

- **Tablas de decisión:** organizar combinaciones de condiciones y acciones cuando existen reglas de negocio múltiples.
- **Transiciones de estado:** comprobar cambios permitidos y prohibidos entre estados.

Ejemplo de aplicación: para un cupo permitido de 1 a 10 entradas, las particiones relevantes incluyen cantidades inferiores a 1, válidas entre 1 y 10, y superiores a 10. Los límites sugeridos son 0, 1, 10 y 11.

Pruebas basadas en estructura / caja blanca

Se derivan de la lógica y la estructura interna del código. Buscan ejercitar sentencias, decisiones, condiciones, caminos y flujos relevantes. La **cobertura** ayuda a identificar partes no ejercitadas, pero **alta cobertura no equivale a ausencia de defectos**.

Complementariedad

La caja negra descubre discrepancias respecto de requisitos; la caja blanca permite examinar recorridos internos. Ninguna sustituye a la otra. También puede recurrirse a experiencia, riesgos y patrones de defectos para priorizar casos.

1.7. Tipos de prueba y pruebas después de cambios

Las **pruebas funcionales** verifican funciones y reglas del negocio. Las **no funcionales** examinan atributos como rendimiento, seguridad, usabilidad, confiabilidad y compatibilidad. El tipo de prueba indica **qué propiedad** se evalúa; el nivel indica **qué alcance** del sistema se examina.

Prueba	Propósito principal
Smoke	Verificación amplia y superficial de que una versión es suficientemente estable para continuar probándola.
Sanity	Comprobación focalizada de que un cambio o funcionalidad concreta funciona de manera razonable.
Reprueba / retesting	Volver a ejecutar el caso que falló para verificar que se corrigió el defecto.
Regresión	Comprobar que los cambios no rompieron comportamientos que antes funcionaban.

Smoke no es sinónimo de sanity y **retesting no es regresión**. Después de corregir un defecto puede ser necesario hacer ambos: verificar la corrección puntual y revisar posibles efectos colaterales.

1.8. Verificación, validación y modelo V

- **Verificación:** comprobar que el producto o entregable se construye de acuerdo con especificaciones, diseños y criterios establecidos: «¿Estamos construyendo el producto correctamente?».
- **Validación:** comprobar que el producto satisface las necesidades reales del usuario o cliente: «¿Estamos construyendo el producto correcto?».

El **modelo V** relaciona actividades de definición y diseño con niveles de prueba: requisitos y aceptación, diseño de sistema y prueba de sistema, diseño de componentes e integración, implementación y pruebas unitarias. Su valor es mostrar que las pruebas se **piensan desde etapas tempranas**, no solo al finalizar la programación.

---

2. Testing en procesos ágiles y desarrollo guiado por pruebas

2.1. Testing ágil

En un desarrollo ágil, la calidad y las pruebas se integran a cada iteración o incremento. Se busca retroalimentación temprana, colaboración entre desarrollo, testing y negocio, automatización donde aporte valor y prevención de defectos. El testing deja de considerarse exclusivamente una etapa final separada.

Las pruebas deben acompañar cambios frecuentes de requisitos, entregas pequeñas y revisión continua del producto. El equipo comparte la responsabilidad por la calidad: no se limita a «pasar el producto a QA» cuando el código está terminado.

2.2. Cuadrantes de testing ágil

Los cuadrantes ayudan a **organizar propósitos de prueba**, no representan una secuencia obligatoria:

Cuadrante	Orientación	Ejemplos
Q1	Apoya al equipo; foco tecnológico.	Unitarias, componentes, automatizadas.
Q2	Apoya al equipo; foco de negocio.	Ejemplos de aceptación, pruebas funcionales, historias y criterios.
Q3	Evalúa el producto; foco de negocio.	Exploratorias, usabilidad, pruebas con usuarios.
Q4	Evalúa el producto; foco tecnológico.	Rendimiento, carga, seguridad y otros atributos técnicos.

Los cuadrantes no asignan una prueba de forma rígida a una sola persona ni sustituyen una estrategia integral.

2.3. TDD — Test Driven Development

TDD es una práctica de desarrollo que comienza con la escritura de una prueba automatizada para una pequeña porción del comportamiento esperado, antes de implementar el código productivo correspondiente.

Ciclo **Red** → **Green** → **Refactor**:

1. **Red:** escribir una prueba que falla porque el comportamiento todavía no está implementado.
2. **Green:** escribir el código mínimo necesario para que pase.
3. **Refactor:** mejorar estructura, legibilidad y diseño **sin cambiar el comportamiento observable**, manteniendo las pruebas en verde.

Se repite en ciclos breves. TDD aporta retroalimentación inmediata, favorece un diseño verificable y construye una red de pruebas de regresión. **No reemplaza** pruebas de integración, sistema, aceptación o exploratorias.

2.4. ATDD y BDD

- **ATDD (Acceptance Test-Driven Development):** define ejemplos y criterios de aceptación con participación del negocio, testing y desarrollo antes de implementar. Orienta el trabajo hacia lo que el cliente considera aceptable.
- **BDD (Behavior-Driven Development):** expresa comportamientos observables mediante ejemplos y lenguaje compartido; puede usar estructuras como **Dado / Cuando / Entonces**.
- **TDD:** se centra especialmente en guiar la implementación técnica mediante pruebas pequeñas y frecuentes.

Se relacionan, pero no son sinónimos: ATDD pone el énfasis en criterios de aceptación; BDD, en comportamiento y comunicación; TDD, en el ciclo de desarrollo guiado por pruebas.

---

3. Lean aplicado al desarrollo de software

3.1. Origen y propósito

Lean se vincula con las prácticas de producción de Toyota y con la búsqueda de **maximizar el valor para el cliente**, optimizar recursos y **eliminar desperdicios**. En software es una filosofía de gestión y mejora continua, no un proceso de desarrollo rígidamente prescrito. Pretende que el trabajo aporte valor, circule con menos obstáculos y se aprenda del resultado.

Valor es aquello por lo que el cliente obtiene un beneficio real; **desperdicio** es trabajo, espera o consumo de recursos que no agrega valor. No se trata de exigir que las personas estén ocupadas permanentemente, sino de optimizar el sistema completo.

3.2. Los siete principios de Lean Software Development

1. **Eliminar desperdicios:** identificar actividades sin valor y reducirlas, evitando documentación, funcionalidades, esperas o transferencias innecesarias.
2. **Amplificar el aprendizaje:** obtener conocimiento mediante iteraciones, retroalimentación, experimentación y mejora continua.

3. **Incorporar la integridad (integridad conceptual):** mantener coherencia de diseño, arquitectura, funcionalidades y experiencia; construir calidad desde el inicio.
4. **Diferir compromisos:** no tomar decisiones irreversibles prematuramente cuando todavía falta información; decidir responsablemente en el momento adecuado.
5. **Dar poder al equipo:** favorecer autonomía, conocimiento, responsabilidad y toma de decisiones cercana al trabajo.
6. **Ver el todo:** optimizar el flujo completo y evitar mejoras locales que perjudiquen al sistema.
7. **Entregar rápido:** acortar ciclos, reducir demoras y obtener retroalimentación real cuanto antes.

Diferir compromisos no significa postergar todo; significa conservar opciones cuando existe incertidumbre. **Entregar rápido no significa sacrificar calidad**: reducir retrabajo y esperas contribuye a entregar mejor.

3.3. Desperdicios en software

La bibliografía relaciona desperdicios de manufactura con situaciones frecuentes en desarrollo:

Desperdicio	Manifestación en software
Trabajo parcialmente terminado	Funcionalidades empezadas que no llegan a estar listas para entregar.
Funcionalidades adicionales	Construir características que nadie necesita o que no generan valor.
Reaprendizaje	Perder conocimiento y tener que redescubrir decisiones o soluciones.
Transferencias de trabajo	Handoffs que generan esperas, pérdida de contexto y malentendidos.
Demoras	Esperar decisiones, pruebas, accesos, aprobaciones o dependencias.
Cambio de tareas / contexto	Interrumpir repetidamente el trabajo para atender muchos frentes.
Defectos	Correcciones, retrabajo y costos de mala calidad.

El material de clase también considera explícitamente el **talento no aprovechado** como desperdicio adicional. Las enumeraciones pueden variar según la fuente; lo central es reconocer qué no aporta valor en el flujo real.

3.4. Lean y Agile

Lean y Agile comparten entrega temprana, aprendizaje, orientación al cliente y adaptación. Lean pone especial énfasis en **flujo, valor, eliminación de desperdicio y optimización integral**. Agile incorpora valores y principios para responder al cambio mediante colaboración y entregas frecuentes. Son enfoques compatibles; **Kanban** permite aplicar muchas ideas Lean al trabajo de software.

---

4. Kanban y métricas

4.1. Qué es Kanban

Kanban es un método para gestionar y mejorar el flujo de trabajo mediante visualización, límites al **trabajo en progreso (WIP)** y un sistema **pull**. Se puede aplicar sobre un proceso existente, sin imponer desde el comienzo una reorganización completa de roles y responsabilidades. Su introducción promueve **cambio evolutivo** y mejora continua.

Un **tablero** representa estados del trabajo y tarjetas que avanzan por ellos. Pero **un tablero con columnas, por sí solo, no constituye necesariamente un sistema Kanban**: el límite de trabajo en curso y las señales de capacidad para traccionar trabajo son fundamentales.

4.2. Principios de gestión del cambio y entrega del servicio

Los materiales distinguen principios de **gestión del cambio** y de **entrega de servicios**:

- **Empezar con lo que se hace ahora:** comprender el proceso real antes de intervenir.
- **Acordar mejoras evolutivas:** evitar imponer transformaciones bruscas sin entender sus consecuencias.
- **Promover liderazgo en todos los niveles:** las oportunidades de mejora pueden surgir de cualquier integrante.
- **Comprender necesidades y expectativas del cliente:** mirar el trabajo como prestación de un servicio.
- **Gestionar el trabajo y permitir que las personas se organicen:** enfocar la mejora en el sistema de trabajo.
- **Revisar la red de servicios y sus políticas:** optimizar resultados y satisfacción del cliente.

4.3. Prácticas y propiedades

La exposición de clase enumera seis prácticas: **visualizar el trabajo; limitar WIP; gestionar el flujo; hacer explícitas las políticas; mejorar colaborativa y evolutivamente; respetar roles, responsabilidades y habilidades existentes**. En la guía de Kanban incluida en las lecturas, la formulación de prácticas incluye **implementar ciclos de retroalimentación** y **mejorar colaborativamente mediante modelos y experimentación**. Son presentaciones de fuentes distintas: conviene reconocer ambas sin atribuirlas a una misma lista literal.

El capítulo «¿Qué es el método Kanban?» de David J. Anderson identifica cinco propiedades centrales: **visualizar el flujo**, **limitar WIP**, **medir y administrar el flujo**, **hacer explícitas las políticas** y **usar modelos para reconocer oportunidades de mejora**. Plantea Kanban como sistema adaptativo complejo y agente de cambio.

4.4. Pull, WIP y cuellos de botella

En un sistema **push**, el trabajo se asigna o empuja hacia la etapa siguiente aunque no tenga capacidad. En **pull**, la etapa siguiente **toma** una nueva unidad cuando puede atenderla. El **límite WIP** establece cuántos ítems pueden estar simultáneamente en un estado o parte del flujo.

Si **Desarrollo** tiene WIP 4 y ya hay cuatro tareas en curso, no se debe iniciar una quinta sin liberar capacidad. Si **Pruebas** acumula tareas, ese cuello de botella puede exigir ayudar a terminar pruebas antes de iniciar más desarrollo. **Optimizar ocupación individual no es igual a optimizar flujo total**.

4.5. Cómo introducir Kanban

1. **Identificar el proceso actual** y sus etapas reales.
2. **Definir la unidad de trabajo** (requisito, historia de usuario, incidencia, tarea).
3. **Representar el flujo** en un tablero y acordar cuándo una tarjeta puede pasar de estado.
4. **Establecer límites WIP** acordes con la capacidad real y ajustarlos con evidencia.
5. **Definir políticas explícitas**, criterios de calidad y reglas de movimiento.
6. **Definir clases de servicio** según urgencia, impacto o compromiso.
7. **Medir el flujo**, observar bloqueos y mejorar colaborativamente.

Las **clases de servicio** permiten tratar de manera diferenciada trabajos urgentes, con fecha fija, estándar o intangibles, según las necesidades del servicio. No deben confundirse con niveles de severidad de defectos.

4.6. Métricas del flujo

Métrica	Qué observa
Lead Time	Tiempo transcurrido desde que se solicita o ingresa un trabajo hasta que se entrega; perspectiva del cliente.
Cycle Time	Tiempo desde que el equipo comienza a trabajar activamente sobre el ítem hasta que se termina; perspectiva del proceso.
Touch Time	Tiempo efectivo de trabajo sobre el ítem, excluyendo esperas.
Eficiencia del ciclo	Proporción de tiempo activo respecto del tiempo transcurrido: `Touch Time / tiempo total del ciclo × 100`.
Throughput / rendimiento	Cantidad de unidades completadas por unidad de tiempo.
WIP	Cantidad de unidades iniciadas pero no terminadas.

Ejemplo de aplicación: si una tarea pasa 10 horas en el flujo, pero recibe 4 horas de trabajo efectivo, su eficiencia del ciclo es `4/10 × 100 = 40 %`. El resto del tiempo no es necesariamente trabajo productivo; puede haber esperas o bloqueos. **Es esencial declarar desde qué evento se mide cada intervalo**, porque distintas fuentes usan convenciones de inicio ligeramente diferentes.

4.7. Métricas de producto, proyecto y proceso

- **Producto:** describen propiedades del software construido, como tamaño, defectos y atributos de calidad.
- **Proyecto (tácticas):** permiten gestionar un proyecto específico, su esfuerzo, calendario, desviaciones y progreso; interesan al equipo y a quien lo lidera.
- **Proceso (organizacionales o estratégicas):** agregan y despersonalizan resultados de múltiples proyectos para observar cómo trabaja la organización y dónde mejorar.

Una misma variable puede analizarse desde diferentes perspectivas: el **tiempo** de un proyecto sirve para gestionar ese proyecto; las desviaciones de tiempo agregadas de muchos proyectos sirven para evaluar el proceso organizacional. El material destaca cuatro mediciones básicas: **tamaño, esfuerzo, tiempo y defectos**.

4.8. Métricas en proyectos ágiles

Velocity (velocidad): cantidad de trabajo estimado que un equipo completa en una iteración, por ejemplo, suma de puntos de historias terminadas. Sirve como referencia de planificación para **ese equipo**, no como medida universal de productividad ni para comparar personas.

Capacidad: cantidad de trabajo que el equipo puede asumir en el próximo período considerando disponibilidad y experiencia. La velocidad histórica puede orientar esta estimación, pero no la determina mecánicamente.

RTF / Running Tested Features: el material lo presenta como una medida de funcionalidades que funcionan y han superado las pruebas. Complementa métricas de avance porque se orienta a **software efectivamente probado**, no solo a tareas declaradas completas.

Métricas de valor y flujo: deben ayudar a tomar decisiones y mejorar, **no a castigar o comparar personas**. El análisis de tendencias es más útil que una cifra aislada.

4.9. Scrum y Kanban: comparación

Aspecto	Scrum	Kanban
Cadencia	Sprints de duración fija.	Flujo continuo; cadencias y períodos de revisión configurables.
Roles	Responsabilidades definidas por el marco Scrum.	No impone roles específicos.
Trabajo en progreso	Limitado indirectamente por el trabajo seleccionado para el Sprint.	Limitado explícitamente mediante WIP por estado o parte del flujo.
Planificación	Objetivo y planificación del Sprint.	Reposición de trabajo según capacidad disponible.
Métricas habituales en el material	Velocidad, avance del Sprint.	Lead Time, Cycle Time, WIP, rendimiento y eficiencia.
Tablero	Refleja el trabajo del Sprint; se renueva su contenido entre Sprints.	Representa un flujo persistente.
Cambios	Se protege el objetivo del Sprint; se renegocia alcance sin ponerlo en riesgo.	Se pueden incorporar nuevos ítems cuando existe capacidad.

Similitudes: ambos se relacionan con Agile/Lean, trabajo visible, colaboración, mejora continua, fragmentación del trabajo, entrega frecuente y decisiones basadas en evidencia. En algunas formulaciones históricas de Scrum, las estimaciones y los gráficos burndown aparecen como obligatorios.

---

5. Calidad de proceso y calidad de producto

5.1. Concepto de calidad

La **calidad** comprende características de un producto o servicio que le permiten satisfacer necesidades **explícitas e implícitas** y generar valor. No depende exclusivamente de que el código funcione: también se relaciona con cumplimiento de compromisos, costos, tiempos, mantenibilidad y experiencia de uso.

La calidad tiene diferentes **perspectivas**: usuario, cliente, equipo de desarrollo, gerencia y otras partes interesadas. El usuario puede priorizar facilidad de uso y rapidez; desarrollo, mantenibilidad y estructura; gerencia, costos, riesgos y cumplimiento. La gestión de calidad busca integrar esas perspectivas.

La bibliografía describe cinco puntos de vista de la calidad: **trascendental**, **del usuario**, **del fabricante**, **del producto** y **basado en el valor**. Ninguno agota por sí solo el concepto.

5.2. Calidad programada, necesaria y lograda

Dimensión	Significado
Calidad programada	Nivel de calidad definido como objetivo durante planificación y diseño.
Calidad necesaria	Calidad requerida para satisfacer necesidades y expectativas reales.
Calidad lograda	Calidad efectivamente alcanzada y observable al evaluar el producto.

El desafío es que estas tres dimensiones **coincidan en la mayor medida posible**. Programar características innecesarias produce desperdicio; lograr menos calidad de la necesaria genera insatisfacción.

5.3. Calidad del proceso frente a calidad del producto

Calidad de proceso: cómo se trabaja para construir software: procedimientos, prácticas, estándares, competencias, medición, auditorías y mejora. **Calidad de producto:** características del resultado obtenido: funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad, seguridad y portabilidad, entre otras.

Una mejora de proceso **puede contribuir** a mejorar el producto, pero la relación no es automática ni simple: en software intervienen decisiones intelectuales, personas, herramientas y contexto. La calidad del producto debe evaluarse también directamente.

5.4. Ideas fundamentales del aseguramiento de calidad

Los apuntes destacan diez orientaciones:

1. La calidad **no se inyecta ni se compra** al final; debe incorporarse desde el inicio.
2. Es una **responsabilidad compartida** por toda la organización.
3. Las **personas y su capacitación** son esenciales.
4. Hace falta **respaldo gerencial** y recursos.
5. Los líderes deben **dar el ejemplo**.
6. El testing aporta **evidencia y medición** de calidad.
7. Conviene **empezar por lo básico** y mantener la simplicidad.
8. El aseguramiento debe **planificarse**.
9. **Más pruebas no garantizan automáticamente más calidad**.
10. El esfuerzo debe ser **razonable y adecuado al negocio**.

5.5. Modelos y atributos de calidad del producto

Los **modelos de calidad** proporcionan estructuras para describir y evaluar atributos del software. Las lecturas incluyen referencias a **McCall**, **ISO 9126** y atributos como funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad. También se mencionan características como seguridad, robustez, modularidad, adaptabilidad, comprensibilidad y facilidad de prueba.

Ejemplo de aplicación: un sistema puede cumplir correctamente una regla de negocio y, sin embargo, tener mala calidad percibida porque responde demasiado lento o es difícil de usar. La evaluación debe cubrir tanto lo funcional como lo no funcional.

5.6. Costos de la calidad

La bibliografía organiza los costos en:

- **Prevención:** actividades para evitar introducir defectos, como formación, estándares y prácticas adecuadas.
- **Evaluación:** actividades para examinar la calidad, como revisiones, inspecciones y pruebas.
- **Fallas internas:** problemas encontrados antes de entregar, que implican corrección y retrabajo.
- **Fallas externas:** problemas descubiertos después de la entrega, con impacto en clientes, soporte, reputación y operación.

Invertir en prevención y evaluación no elimina todos los costos, pero ayuda a reducir los asociados con fallas, especialmente las externas.

---

6. Aseguramiento de calidad y mejora de procesos

6.1. QA, QC y gestión de calidad

Gestión de calidad: planificación, coordinación, control y mejora de las actividades relacionadas con calidad. **Aseguramiento de calidad (QA/SQA):** actividades planificadas y sistemáticas orientadas a proporcionar confianza en que los procesos y productos cumplirán requisitos. **Control de calidad (QC):** comprobaciones y evaluaciones de resultados para identificar desviaciones y defectos.

En la explicación de la bibliografía, SQA aporta **infraestructura, auditorías y reportes** para evaluar la eficacia de las actividades de control de calidad. Su papel no es simplemente ejecutar pruebas. Los hallazgos deben llegar a quienes pueden asignar recursos y corregir problemas.

6.2. Aseguramiento de calidad de procesos y productos

El **aseguramiento de proceso** se ocupa de establecer estándares, procedimientos, capacitación, medición y mecanismos de revisión. El **aseguramiento de producto** comprueba que los entregables se ajusten a requisitos y estándares mediante revisiones, inspecciones, pruebas y evaluaciones. Ambos se complementan.

Los **estándares** permiten un trabajo consistente y hacen más sencillo revisar resultados, aunque deben adaptarse al contexto y aportar valor. La bibliografía presenta **ISO 9001** como un estándar de sistemas de gestión de calidad, no como una receta específica de programación.

6.3. Plan y grupo de SQA

Un **plan de aseguramiento de calidad** identifica objetivos, estándares aplicables, actividades, responsables, recursos, registros, revisiones, auditorías y mecanismos de reporte. El grupo o función de SQA puede verificar la adhesión a procesos y estándares, documentar no conformidades y elevar resultados a responsables técnicos y gerenciales.

Los reportes deben registrar hallazgos verificables y dar seguimiento a acciones correctivas. La independencia de la revisión puede ayudar a identificar problemas que un equipo involucrado directamente en la ejecución podría pasar por alto.

6.4. Mejora continua de procesos

Mejorar procesos significa **analizar cómo se trabaja, medir resultados, identificar problemas, introducir cambios y comprobar su efecto**. El material bibliográfico de Sommerville desarrolla un ciclo de **medición, análisis y cambio de procesos**, con nueva medición posterior para verificar si hubo mejora.

La mejora no consiste únicamente en documentar más: debe traducirse en mejores resultados, menos defectos, mayor previsibilidad, menor desperdicio o mayor satisfacción del cliente. Los datos deben interpretarse dentro de su contexto.

6.5. Medición orientada a objetivos

El enfoque **GQM (Goal–Question–Metric / Meta–Pregunta–Métrica)** parte de una meta, formula preguntas que permitan evaluar su cumplimiento y selecciona métricas capaces de responderlas.

Ejemplo de aplicación: meta: disminuir retrasos en entregas. Pregunta: ¿cuánto se desvían los proyectos respecto del calendario comprometido? Métrica: desviación porcentual de fechas y distribución de retrasos. No se debe empezar midiendo por medir: primero se define **qué decisión se quiere mejorar**.

Las mediciones pueden ser ambiguas o engañosas cuando se comparan proyectos distintos sin normalización, se usan definiciones diferentes o se incentiva optimizar una cifra aislada.

6.6. CMMI — mejora de procesos

CMMI (Capability Maturity Model Integration) es un modelo de referencia para orientar la mejora de procesos. El material presenta dos enfoques de representación:

- **Por etapas / madurez:** evalúa la madurez del conjunto de procesos de la organización.
- **Continua / capacidad:** permite focalizar la mejora en áreas de proceso particulares.

Niveles de madurez presentados en el material

Nivel	Descripción
1. Inicial	Procesos poco predecibles, dependientes del esfuerzo individual.
2. Gestionado	Gestión básica de proyectos y compromisos; planificación y seguimiento.
3. Definido	Procesos organizacionales documentados, estandarizados y adaptables.
4. Gestionado cuantitativamente	Control y comprensión del desempeño mediante mediciones cuantitativas.
5. Optimizado	Mejora continua basada en análisis, aprendizaje e innovación.

Madurez describe el estado de los procesos organizacionales; **capacidad** permite observar el desempeño y mejora de procesos concretos. No deben confundirse.

6.7. CMMI y enfoques ágiles

CMMI se enfoca en **qué capacidades y resultados de proceso** debe desarrollar una organización; los enfoques ágiles proponen valores, principios y prácticas para trabajar adaptativamente. No son necesariamente excluyentes. El desafío consiste en evitar que la formalización burocratice innecesariamente el trabajo y, a la vez, mantener evidencia, trazabilidad y disciplina suficientes para sostener la calidad.

---

7. Revisiones técnicas, walkthroughs e inspecciones

7.1. Qué son las revisiones técnicas

Las **revisiones técnicas** son actividades de evaluación de artefactos de software por otras personas, destinadas a detectar errores, omisiones, inconsistencias y problemas de calidad **sin necesidad de ejecutar el programa**. Pueden aplicarse a requisitos, diseños, código, documentación, planes y otros entregables.

Son una forma de **verificación estática** y complementan el testing dinámico. Detectar un problema antes de que avance a etapas posteriores reduce retrabajo y costo. También favorecen la transferencia de conocimiento y la uniformidad de criterios.

7.2. Revisión informal, walkthrough e inspección

Técnica	Características
Revisión informal	Evaluación flexible, con poca estructura; útil para detectar problemas rápidamente.
Walkthrough / recorrido	El autor o responsable guía a otras personas por el artefacto para examinarlo y recibir observaciones.
Inspección formal	Revisión sistemática con preparación, roles, criterios y registro de defectos; busca identificar problemas de manera rigurosa.

En las **inspecciones**, el foco es encontrar defectos del artefacto, **no evaluar a la persona que lo produjo**. Los resultados deben registrarse, corregirse y verificarse posteriormente.

7.3. Etapas y participantes de una inspección

Una secuencia típica comprende **planificación**, **presentación o preparación del material**, **lectura individual**, **reunión de inspección**, **retrabajo** y **seguimiento**. Pueden intervenir autor, moderador, lectores o inspectores y registrador, según la formalidad del proceso.

Los criterios de entrada, listas de comprobación y estándares ayudan a que la revisión sea reproducible. Es importante que el artefacto esté suficientemente preparado: una reunión que se dedica a descubrir problemas obvios de formato desperdicia tiempo de revisión técnica.

7.4. Revisiones frente a pruebas

Revisiones	Pruebas
Analizan artefactos sin ejecutarlos.	Ejecutan el software o parte de él.
Pueden comenzar desde requisitos y diseño.	Requieren un objeto ejecutable o comprobable según el nivel.
Detectan errores, omisiones y problemas de estructura.	Detectan discrepancias entre comportamiento observado y esperado.
Favorecen prevención y detección temprana.	Aportan evidencia del comportamiento en ejecución.

No son alternativas excluyentes: **revisiones + testing** ofrecen una estrategia de calidad más completa.

7.5. Métricas y resultados de revisión

Las revisiones pueden registrar cantidad y tipo de defectos, esfuerzo dedicado, tasa de detección, distribución por artefacto y reincidencia. Su utilidad está en identificar causas recurrentes y mejorar procesos. La métrica aislada **«defectos encontrados por persona»** puede incentivar comportamientos indeseables y no debe usarse para juzgar individualmente.

---

8. Cuadros integradores para el parcial

8.1. Conceptos que suelen confundirse

Conceptos	Diferencia fundamental
Error / defecto / falla	Equivocación, problema incorporado al artefacto y manifestación observable en ejecución, respectivamente, con los matices de la terminología de clase.
Severidad / prioridad	Gravedad del defecto / urgencia de corrección.
Nivel / tipo / ambiente de prueba	Alcance / propiedad evaluada / entorno de ejecución.
Smoke / sanity	Comprobación general inicial / comprobación focalizada.
Retesting / regresión	Confirmar corrección / comprobar que no se rompió lo existente.
Verificación / validación	Cumplir especificaciones / satisfacer necesidades reales.
TDD / ATDD / BDD	Guía técnica por pruebas / aceptación colaborativa / ejemplos de comportamiento.
Lean / Kanban	Filosofía de valor y desperdicios / método de gestión y mejora del flujo.
Push / pull	Empujar trabajo sin considerar capacidad / tomarlo cuando hay capacidad.
Lead Time / Cycle Time / Touch Time	Tiempo desde solicitud / tiempo desde inicio del trabajo / tiempo activo.
Producto / proyecto / proceso	Calidad del resultado / gestión de un esfuerzo concreto / desempeño organizacional.
QA / QC / testing	Confianza y sistema de aseguramiento / evaluación y control / pruebas de comportamiento.
Revisión / inspección / testing	Evaluación estática / revisión formal sistemática / evaluación dinámica.
CMMI madurez / capacidad	Conjunto organizacional de procesos / áreas de proceso específicas.

8.2. Relaciones entre los siete temas

La relación general del parcial puede leerse así:

1. **Testing** proporciona evidencia sobre defectos y comportamiento.
2. **Testing ágil y TDD** incorporan esa evidencia al ciclo de desarrollo desde temprano.
3. **Lean** ayuda a eliminar actividades sin valor y mejorar la entrega.

4. **Kanban** hace visible y medible el flujo para introducir mejoras evolutivas.
5. **Calidad de producto y proceso** establece qué significa construir y entregar bien.
6. **SQA, medición y CMMI** institucionalizan la prevención, evaluación y mejora.
7. **Revisiones técnicas** detectan errores antes de la ejecución y complementan las pruebas.

Idea integradora: la calidad no aparece al final de un proyecto ni depende de una sola actividad. Se construye mediante decisiones de diseño, prácticas de desarrollo, pruebas, revisiones, procesos adecuados, medición y mejora continua.

8.3. Preguntas de repaso sugeridas

1. ¿Por qué el testing no puede demostrar la ausencia total de defectos?
2. ¿Qué diferencia existe entre severidad y prioridad? Dar un caso en el que no coincidan.
3. ¿Cómo se relacionan los cuatro niveles de prueba con el modelo V?
4. ¿Por qué la partición de equivalencia y los valores límite reducen esfuerzo?
5. ¿Qué diferencia hay entre smoke, sanity, retesting y regresión?
6. ¿Qué diferencia existe entre TDD, ATDD y BDD?
7. ¿Cuáles son los siete principios Lean y qué desperdicios intentan reducir?
8. ¿Por qué un tablero con tarjetas no basta para afirmar que existe Kanban?
9. ¿Cómo se identifica un cuello de botella a partir de WIP y tiempos de flujo?
10. ¿Por qué Scrum limita WIP indirectamente y Kanban explícitamente?
11. ¿Qué cambia al pasar de métricas de proyecto a métricas de proceso?
12. ¿Cómo se distinguen calidad programada, necesaria y lograda?
13. ¿Qué actividades pertenecen a SQA y cuáles son de control o testing?
14. ¿Cómo se aplica GQM para elegir métricas útiles?
15. ¿Cuál es la diferencia entre madurez y capacidad en CMMI?
16. ¿Por qué conviene revisar requisitos y diseño antes de probar el código?
